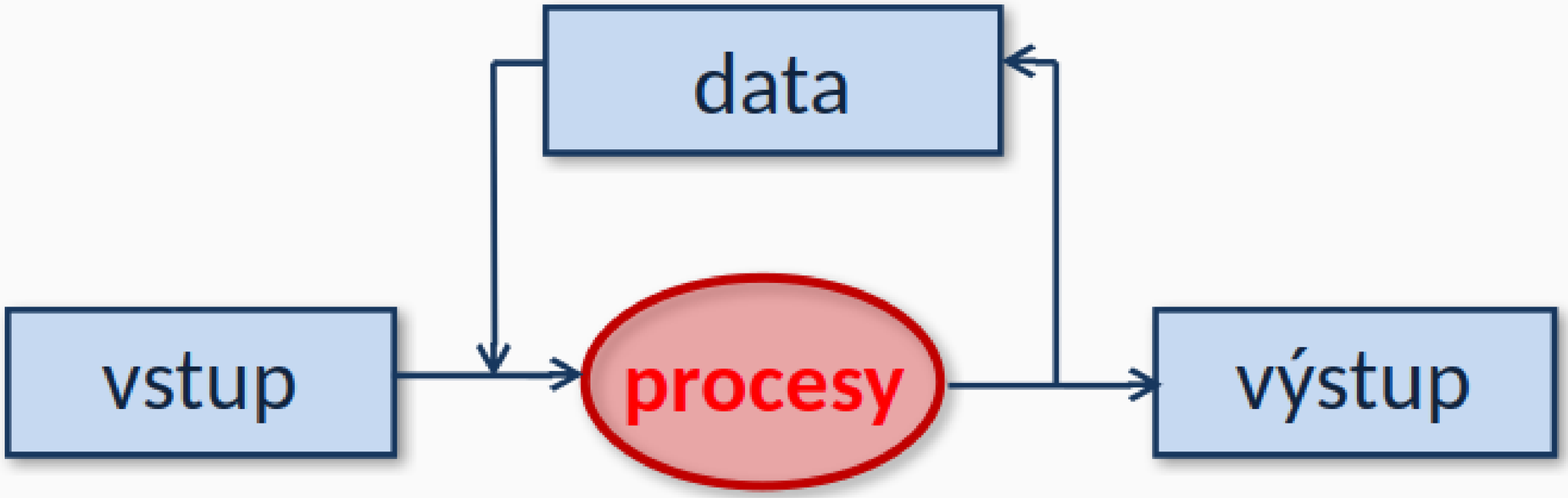
Pokročilé informační systémy

Procesy, pokročilé modely procesů, cesta k workflow

doc. Ing. Radek Burget, Ph.D.
burgetr@fit.vutbr.cz

Procesy ve schématu IS

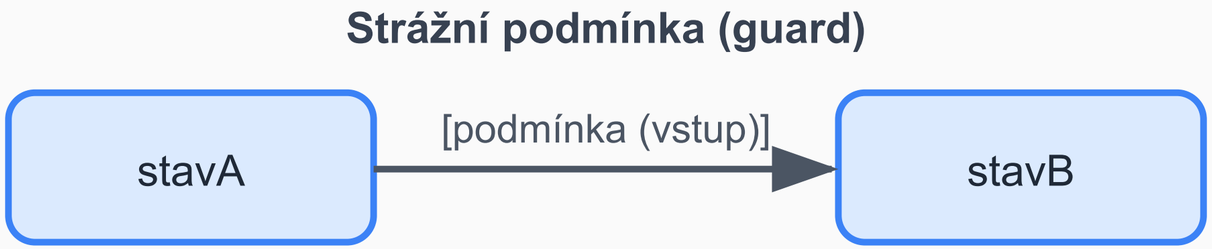


- **Data** uchovávající **stav** systému
- **Procesy** realizující transformace často ve formě *transakcí*.
- Obojí má být **isomorfní** s reálným systémem

Procesy a jejich definice

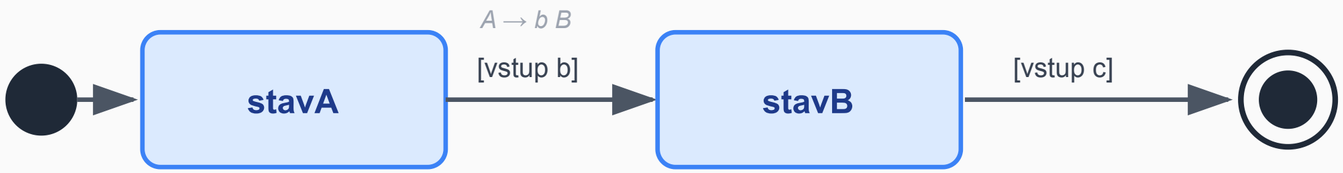
UML stavové diagramy, typy procesů, dvouúrovňové schéma

UML stavový diagram



Sekvenční procesy

- Model = **regulární gramatiky**, **konečné automaty**
  - Pravidlo tvaru A → b B nebo A → b *(A, B jsou stavy; b je vstup – událost nebo podmínka)*
- Stavový diagram popisuje přechody mezi stavy na základě vstupů
- Vstup může být i prázdný



Hierarchické procesy

- Model = **bezkontextové gramatiky**, **zásobníkový automat**
  - Pravidlo tvaru `A → b B pokračováníA`
- Stav může být *vnitřně strukturován* (vnořené stavové stroje)
- Po ukončení vnořeného stavu B se pokračuje v A
- Vstup **musí být vždy označen**, jinak může vzniknout nekonečný cyklus
- Vnitřní kontext zachován pomocí zásobníku (**H** = history)

Obecné procesy

- Model = **neomezené gramatiky**, **Turingův stroj**
  - Pravidlo tvaru $a \quad A \quad b \rightarrow c \quad B \quad d$
- Stavový diagram neexistuje – popis v programovacím jazyce
- Pokud za paměť považujeme data IS v databázi → **obecné procesy**

Typy procesů – přehled

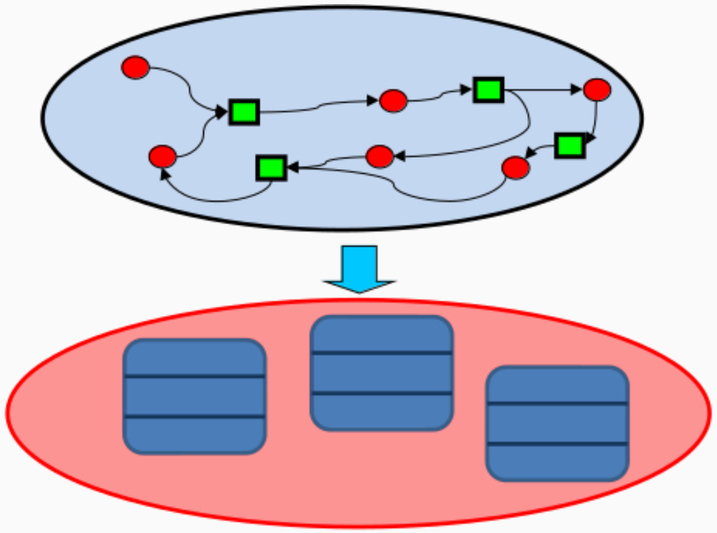
Dvouúrovňové schéma

- Řízení na nejvyšší úrovni: **sekvenční a hierarchické procesy**
  - Popis přechodů mezi stavy (státový diagram)
- Obecné procesy v jednotlivých stavech: popsány **transakcemi**
  - Pracují s databází, mění stav systému

```
Vrstva řízení:    stavový diagram (sekvence, hierarchie)
      ↓
Vrstva stavů:     obecné procesy / transakce (v každém stavu)
```

Dekompozice - dvouúrovňové schéma

- Standardně jsou procesy dekomponovány na řízení a činnost stavů – transakce
- Na nejvyšší úrovni vystačíme se sekvenčními a hierarchickými procesy
- Obecné procesy ve stavech (pracující s databází) – modelujeme v obecném programovacím jazyce (transakce).



řízení - sekvence, hierarchie

dekompozice stavů

stavy - obecný proces někdy ***transakce***

Transakční zpracování

Opakování a rozšíření — ACID, modely transakcí

ACID — přehled

Vlastnost	Význam	Kdo zajišťuje
A – Atomičnost	Celá transakce, nebo nic; commit / rollback	TPS
C – Konzistence	DB splňuje IO před i po transakci	**Programátor**
I – Izolovanost	Souběžné provádění = sekvenční výsledek; úrovně: serializable → repeatable read → read committed → read uncommitted	TPS
D – Trvanlivost	Potvrzené změny přežijí havárii; žurnál, zrcadlení	TPS

Kdo co zajišťuje

- **Programátor** zodpovídá za konzistenci transakce — správně sestavit operace tak, aby DB přešla z jednoho konzistentního stavu do druhého
- **TPS** zajišťuje automaticky:
  - **Atomičnost** – mechanismus commit/rollback
  - **Izolovanost** – zamykání záznamů, uspořadatelné plány
  - **Trvanlivost** – žurnál (write-ahead log), zrcadlení

Celý zbytek přednášky se zabývá modely, kde se záměrně od ACID odchylujeme — vždy je důležité vědět, která vlastnost se porušuje a proč.

Důvody zrušení transakce

Nepředvídatelné:

- havárie systému

V režii TPS:

- porušení integritního omezení
- porušení izolovanosti souběžných transakcí
- detekované uváznutí (deadlock)

V režii transakce / programátora:

- na požadavek samotné transakce (`rollback`)

Modely procesů (transakcí)

Plochá transakce, body návratu, dvouúrovňová dekompozice

Plochá transakce (Flat transaction)

- Minimální model bez vnitřní struktury – obecný proces v programovacím jazyce
- Struktura:
  1. `begin_transaction()`
  2. tělo transakce (sekvence operací)
  3. `commit()` (úspěch) nebo `rollback()` / `abort()` (neúspěch / porucha)
- **Žádné** částečné zotavení ani body návratu

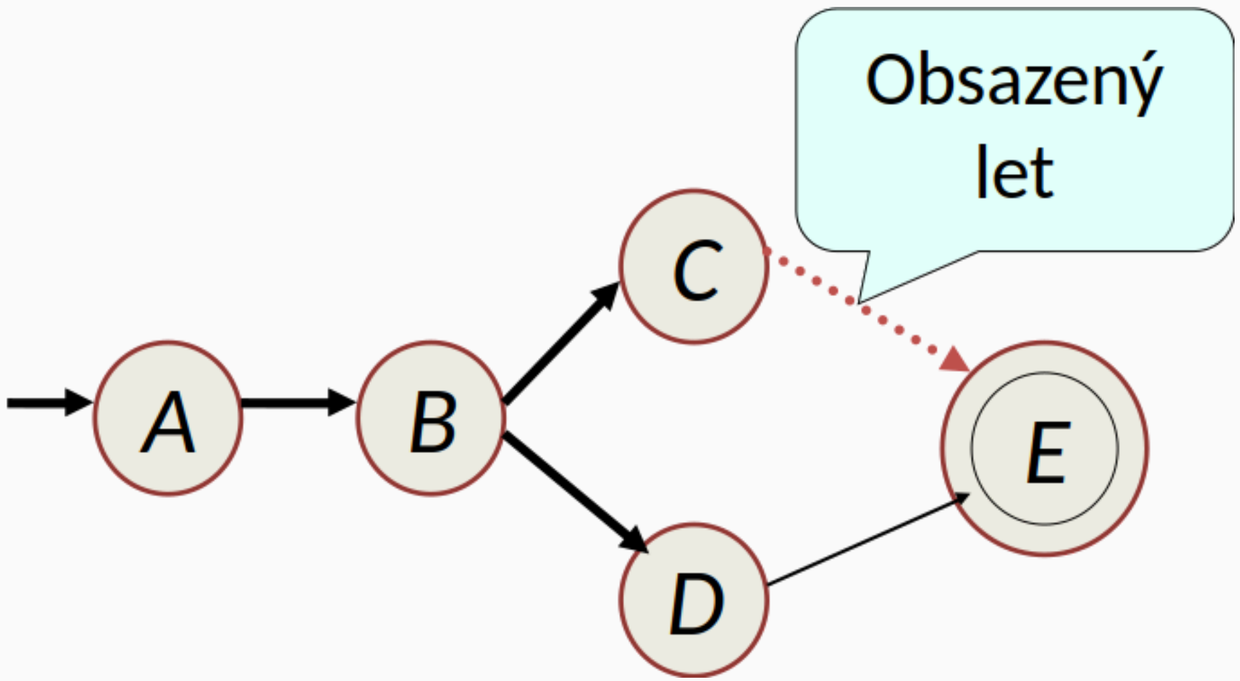
Struktura ploché transakce

```
begin_transaction();

    … blok programu …
    if (chyba) abort();
    …

commit();
```

- Lokální nedatabázové proměnné jsou součástí programu, ne DB kontextu

Problém ploché transakce

- Příklad: rezervace letu A → B → C → E
  - transakce rezervuje A→B, poté B→C
  - zjistí, že C→E není volné
  - **nemůže provést částečný návrat** (jen na B) – musí zrušit celou transakci
  - přijde o práci (rezervaci A→B)
- Řešení: **body návratu (savepoints)**



Body návratu (Savepoints)

- **Savepoint** = bod návratu pro *částečný rollback* v rámci jedné transakce
- Vytvoření: `sp := create_savepoint()`
- Částečný návrat: `rollback(sp)` – obnoví DB kontext, **lokální proměnné zachovány**
- Po rollback se **pokračuje** za příkazem návratu
- Zrušené milníky mezi cílem a místem návratu zanikají

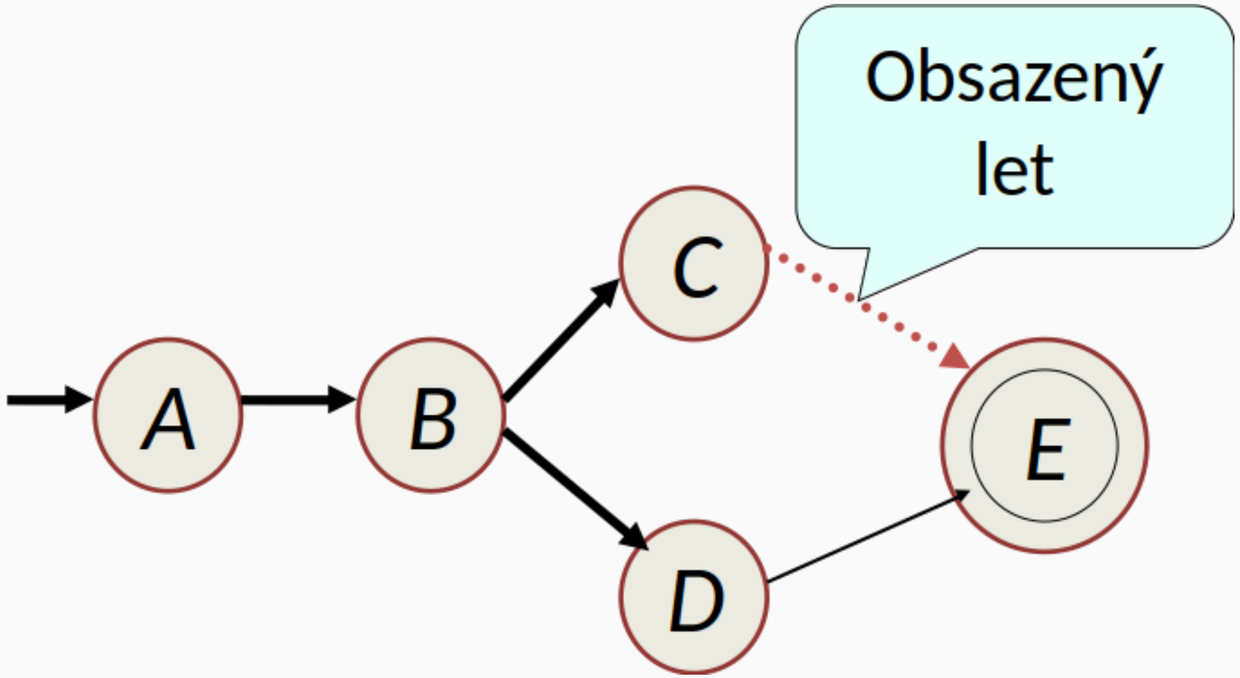
Příklad bodů návratu

```
begin_transaction();
    S1;
    sp1 := create_savepoint();
    S2;
    sp2 := create_savepoint();
    S3;
    if (podmínka) {
        rollback(sp2);   // vrátí S3, pokračuje dál
        S4;
    }
commit();
```

- Po `rollback(sp2)` milník `sp3` již neexistuje
- `abort` vs `rollback`: abort *zastaví* transakci, rollback *pokračuje*

Příklad: rezervace s body návratu

- Na začátku a v každém větvení se vytvoří savepoint
- Při neúspěchu `rollback` na předchozí savepoint → zkouší jinou variantu
- Strategie **backtracking** (zásobník bodů návratu):
  - A→B (sp1), B→C (sp2), C→E obsazeno → rollback(sp2), B→D (sp3), D→E volné → commit



Transakce v Jakarta EE

JTA, @Transactional, @TransactionAttribute — propojení teorie s praxí

JTA — Java Transaction API

- Standardní Jakarta EE API pro správu transakcí (`jakarta.transaction`)
- Definuje rozhraní mezi aplikací, kontejnerem a TPS (XA protokol)
- Tři úrovně správy transakcí:
  - **CMT** (Container-Managed Transactions) — implicitní, zajišťuje kontejner pro EJB
  - **BMT** (Bean-Managed Transactions) — explicitní `UserTransaction`
  - **CDI interceptor** — anotace `@Transactional` od Jakarta Transactions 2.0
- Viz EJB s CMT v přednášce p04

@Transactional — deklarativní správa

```
@ApplicationScoped
public class ObjednavkaService {

    @Inject
    EntityManager em;

    @Transactional
    public void vytvorObjednavku(Objednavka o) {
        em.persist(o);
        // kontejner: begin před metodou, commit po návratu
        // RuntimeException → rollback automaticky
    }
```

- CDI bean (ne EJB) může mít transakci díky `@Transactional`

@TransactionAttribute — řízení propagace

Hodnota	Chování volané metody
`REQUIRED`	Připojí se k existující transakci; není-li, vytvoří novou
`REQUIRES_NEW`	Vždy vytvoří novou transakci; stávající pozastaví
`MANDATORY`	Musí existovat transakce; jinak výjimka
`SUPPORTS`	Připojí se k existující; bez transakce pracuje bez ní
`NOT_SUPPORTED`	Vždy mimo transakci; stávající pozastaví
`NEVER`	Nesmí být aktivní transakce; jinak výjimka

REQUIRES_NEW — nezávislá podtransakce

```
@Transactional   // REQUIRED
public void zpracujObjednavku(long id) {
    ulozObjednavku(id);       // v téže transakci
    odesliPotvrzeni(id);      // nová nezávislá transakce
}

@Transactional(Transactional.TxType.REQUIRES_NEW)
private void odesliPotvrzeni(long id) {
    // commit/rollback tohoto volání neovlivní volající transakci
}
```

- Každé volání je samostatnou podtransakcí (viz dále)
- Neúspěch `odesliPotvrzeni()` nemusí zrušit celou objednávku
- Kontejner provede `chain()` automaticky namísto explicitního volání v aplikaci

EJB a Container-Managed Transactions

- EJB (`@Stateless`, `@Stateful`, `@Singleton`) mají CMT jako výchozí chování
- Každá metoda je implicitně `REQUIRED` — kontejner spustí transakci automaticky
- `@TransactionAttribute` na metodě přebíjí výchozí nastavení

```
@Stateless
public class SkladBean {
    // implicitně REQUIRED na každé metodě

    @TransactionAttribute(TransactionAttributeType.REQUIRES_NEW)
    public void zarezervujZbozi(long polozkaId) { … }
}
```

- Zodpovědnost za **konzistenci** dat zůstává na programátorovi — viz ACID

Shrnutí: transakce v Jakarta EE

Teorie	Jakarta EE
Plochá transakce	`@Transactional` / CMT — kontejner zajistí begin/commit/rollback
`chain()` zřetězená transakce	`REQUIRES_NEW` — kontejner spustí nezávislou podtransakci
XA protokol (dále)	JTA koordinuje více zdrojů (DB + JMS broker) v jedné transakci
Konzistence = zodpovědnost programátora	Anotace nenahrazují správný návrh logiky

Zřetězené transakce

Chained transactions – dvouúrovňová dekompozice sekvencí

Motivace

- Většina aplikací = sekvence transakcí
  - Příklad objednávkového systému: přijetí objednávky → doprava → fakturace
- **Dlouho trvající transakce** snižují výkonnost
  - blokují záznamy, při havárii se ztratí veškerá provedená práce
- Řešení: **dekompozice** na menší podtransakce

Slučování řízení a transakcí

```
přijetí objednávky → [bez podmínky] → doprava → [bez podmínky] → fakturace
```

- Sekvence stavů s přechody bez podmínek → jedna modelovaná struktura
- Dochází ke slučování vrstev dvouúrovňového modelu
  - **Vrstva řízení** (stavový diagram) nerozhoduje nic — jen sekvencuje kroky
  - **Vrstva stavů** (transakce) vykonává obsah každého kroku
- Pokud přechody nemají podmínky, obě vrstvy lze vyjádřit jednou strukturou: **zřetězenou transakcí**

Zřetězená transakce

Schéma zřetězené transakce

```
begin_transaction();
    S1;
commit();        // ST1 potvrzena, trvanlivá
    S2;
commit();        // ST2 potvrzena, trvanlivá
    …
    Sn;
commit();        // STn potvrzena
```

- S_i je tělo i-té transakce, kterou budeme dále označovat jako podtransakci ST_i.
- Každý `commit()` zajišťuje **trvanlivost předchozí podtransakce**
- Při havárii v ST_i zůstanou ST_1, ..., ST_{i-1} zachovány

Vlastnosti vs. ACID

Vlastnost	Zřetězená transakce
Atomičnost	❌ pouze každá podtransakce zvlášť
Konzistence	⚠️ vyžadována po každém commitu
Izolovanost	❌ každá podtransakce izolovaná, celek nikoli
Trvanlivost	✅ potvrzené podtransakce jsou trvanlivé

Databázový kontext mezi podtransakcemi

- Po commitu ST_i se uvolní všechny zámky a kurzory
- Souběžné transakce mají přístup k dříve zamknutým záznamům
- ST_{i+1} může pracovat s jinými hodnotami (změněnými souběžnou transakcí)
- **Každá podtransakce je izolovaná, ale celek není**
- Výhoda: kratší doba zamykání → vyšší propustnost

Komunikace mezi podtransakcemi

- **Lokální proměnné** nepřežijí havárii (ztratí se)
- Alternativa: komunikace přes **databázové proměnné**
  - Před commitem ST_i se uloží do DB → přežijí pád systému
  - Nevýhoda: viditelné ostatním souběžným transakcím → nutnost **zámků**

Kompenzující transakce

- Po potvrzení několika podtransakcí nelze provést klasický rollback
  - Fyzické obnovení by porušilo trvanlivost jiných transakcí
- Řešení: **kompenzující transakce** – logická obnova efektu jiné transakce
- Příklad: registrace studenta do předmětu
  - Kompenzace = odregistrace (sníží počet studentů zpět)
  - Funguje správně i při souběžné registraci dalšího studenta

Alternativní sémantika – chain()

```
begin_transaction();
    S1;
chain();        // potvrdí ST1, začne ST2, ale neuvolní kontext
    S2;
chain();        // potvrdí ST2, začne ST3, neuvolní kontext
    …
    Sn;
commit();
```

- `chain()` **neuvolňuje databázový kontext** (zámky, kurzory)
- Souběžné transakce nevidí mezistavy
- Celek je **izolovaný** – na úkor výkonnosti

Dopředný návrat (roll forward)

- Zotavení z havárie při alternativní sémantice:
  - Klasický rollback nelze – nekonzistentní stav po ST_i
  - Řešení: **dopředný návrat** (roll forward)
- Po restartu TPS se obnoví DB kontext a **dokončí se** nedokončené podtransakce
- Výsledek: zachována izolovanost i atomičnost celé zřetězené transakce

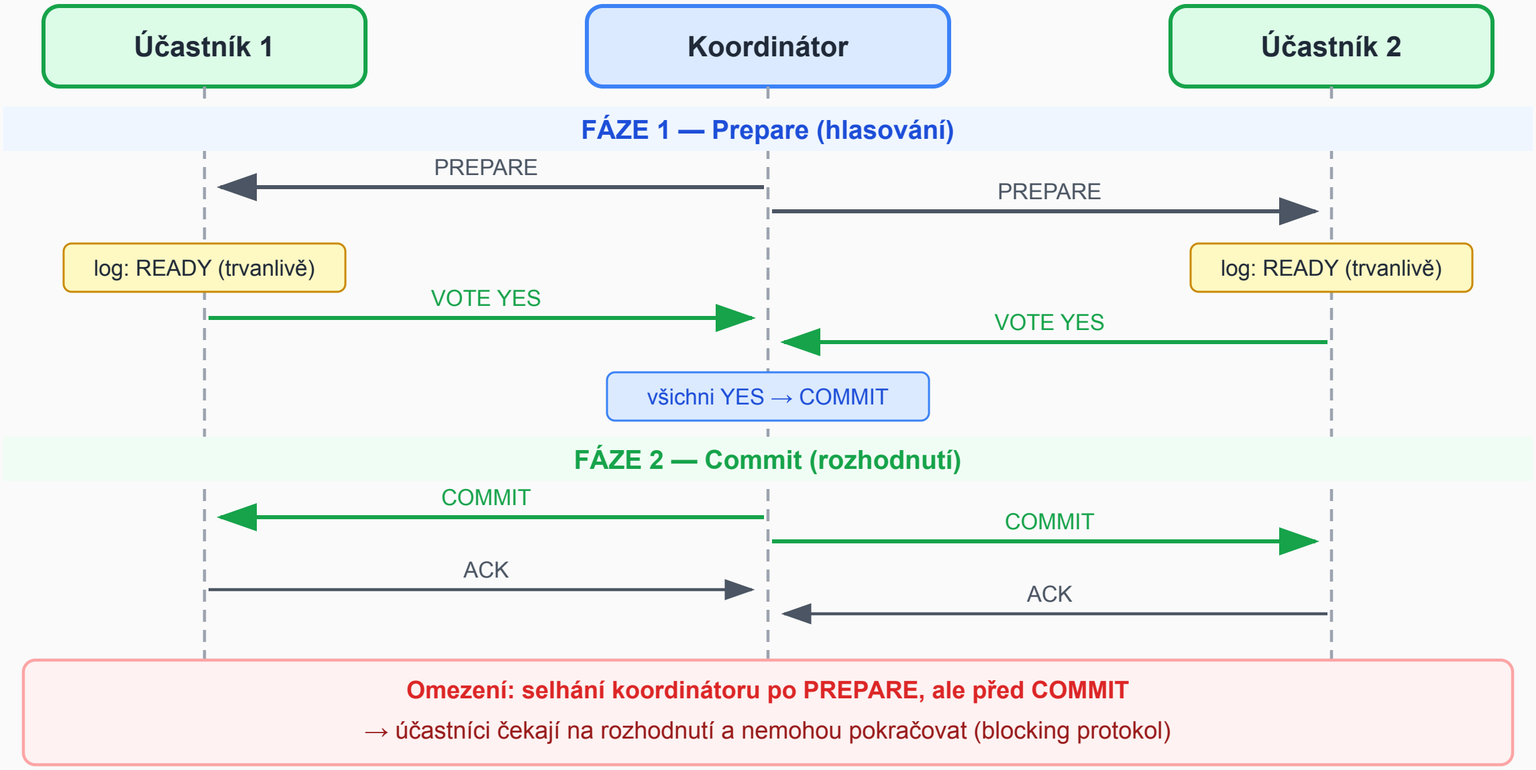
Distribuované transakce

Dvoufázový commit, SAGA pattern

Problém distribuovaných transakcí

- Dosud uvažovaný model: **jeden TPS, jedna databáze**
- Distribuované systémy: více **nezávislých TPS** musí jednat atomicky
  - Příklad: převod peněz mezi dvěma bankami – každá má vlastní TPS
  - Příklad: objednávka v e-shopu spouští sklad, platbu i dopravu jako samostatné služby
- Scénář selhání:
  - Banka A odečte peníze (commit) → Banka B havaruje → peníze jsou v prázdnu
- Klasický rollback nefunguje přes hranice TPS — nutný **protokol pro distribuovaný commit**

Dvoufázový commit (2PC)



- Před hlasováním každý účastník provede svoji část až po event. `commit()`

Dvoufázový commit — vlastnosti

- **Zaručuje atomičnost** přes více nezávislých TPS
- Standard: **XA protokol** (Java JTA, většina enterprise databází)
- **Omezení — blocking protokol:**
  - Koordinátor havaruje *po* Prepare, *před* Commit
  - Účastníci uzamknou záznamy a nemohou pokračovat — čekají neomezeně
  - 3PC (three-phase commit) částečně řeší blokování, ale v praxi se nepoužívá
- **Problém škálovatelnosti:** drží distribuované zámky → nevhodné pro mikroslužby

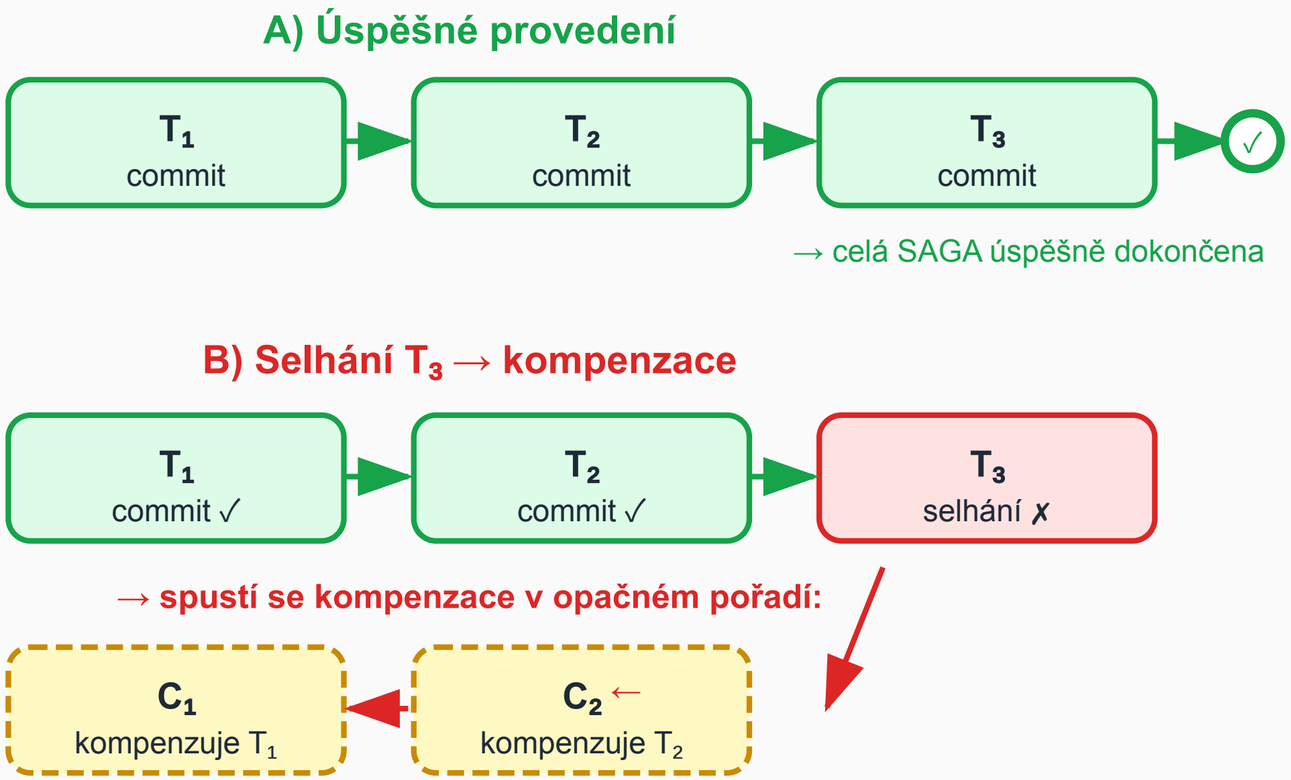
XA protokol

- Standard API koordinaci distribuované transakce (The Open Group)
- Každý **resource manager** (DB, message broker) musí implementovat tři operace:

Operace	Fáze 2PC	Význam
`xa_prepare()`	Prepare	Připrav se, zapiš do logu, vrať YES/NO
`xa_commit()`	Commit	Potvrď transakci
`xa_rollback()`	Rollback	Odvolej transakci

- **Koordinátor** (v Javě JTA) volá tyto operace na všech účastnících
- Databáze i JMS broker v jedné transakci → atomické „ulož do DB + vlož do fronty"

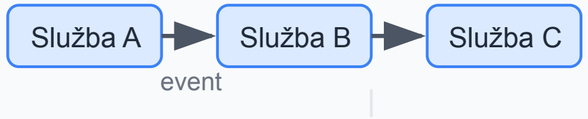
SAGA pattern



Typy orchestrace

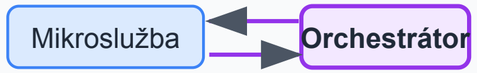
Choreografie (události)

Každá služba naslouchá událostem a samostatně rozhoduje o své akci. Žádný centrální koordinátor.



Orchestrátor

Centrální orchestrátor vydává příkazy a sleduje výsledky.



SAGA vs. 2PC

SAGA = škálovatelnost + eventual consistency | 2PC = silná konzistence + blokování + horší škálovatelnost

SAGA — vlastnosti a použití

- **SAGA** = sekvence lokálních transakcí $T_1, T_2, \ldots, T_n$
  - Každá T_i má svou **kompenzační transakci** C_i (logická obnova efektu)
  - Selhání T_k spustí kompenzace $C_{k-1}, \ldots, C_1$ v opačném pořadí
- Přímá vazba na **kompenzující transakce** (viz předchozí sekce)
  - SAGA je jejich formalizace pro distribuované systémy
- Standardní pattern pro distribuované business transakce v **mikroslužbách**
- Vlastnosti
  - Žádné distribuované zámky (škálovatelnost)
  - Pouze **eventual consistency**
    - Systém je dočasně nekonzistentní mezi T_i a T_{i+1}
  - Kompenzace musí být pečlivě navrženy (ne vždy jde o prostý rollback)

Zotavitelné fronty a reálné události

Recoverable queues, real-world events

Motivace: zotavitelné fronty

- Aplikace nevyžaduje úzké semknutí akcí do jedné izolované transakce
- Stačí zaručit, že po dokončení jedné akce **bude někdy provedena další**
- Na rozdíl od zřetězených transakcí může být mezi akcemi **podstatná časová prodleva**
- Příklad: objednávka → expedice → fakturace
  - Fakturace a expedice mohou proběhnout kdykoli po úspěšné objednávce

Zotavitelná fronta – definice

- **Zotavitelná fronta** = mechanismus pro plánování transakcí k budoucímu vykonání
- Základní operace:
  - **Vlož** – transakce vloží záznam o práci při svém commitu
  - **Vyber** – jiná transakce záznam vyzvedne a práci provede (obvykle spouští server)
- Záznam obsahuje popis akce a data potřebná pro předání (např. ID objednávky)

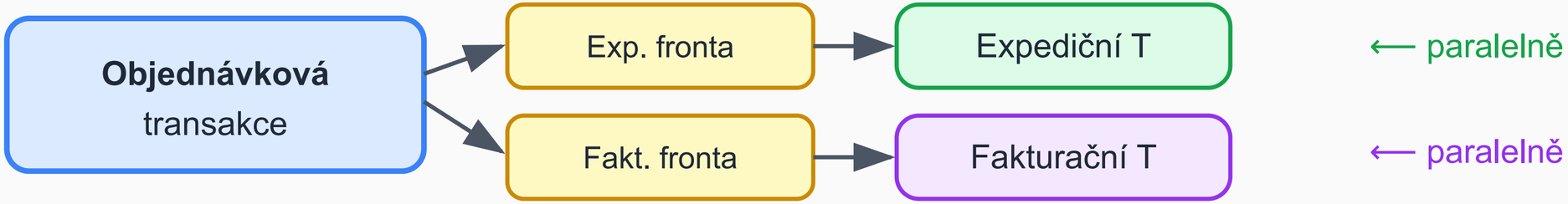
Vlastnosti zotavitelné fronty

- Fronta **musí být trvanlivá** – přežít havárii systému
- Koordinace s transakcemi:
  - Vloží-li transakce do fronty záznam a později je zrušena (rollback), tak musí být tento záznam z fronty odstraněn.
  - Vybere-li transakce z fronty záznam a později je zrušena (rollback), tak musí být tento záznam do fronty navrácen.
  - Záznamy od nepotvrzené transakce nesmí vybírat jiné transakce.

Organizace zotavitelné fronty

Řetěz s pevným uspořádáním (pipeline)

Podpora paralelismu



Paralelismus přes fronty

- Objednávková transakce vloží záznamy do **obou** front najednou
- Expediční a fakturační transakce mohou proběhnout **paralelně**
  - Zákazník může dostat fakturu dříve než zboží (obchodně v pořádku)
- Fronta odděluje producenta od konzumenta v čase i prostoru
  - Nemusí běžet současně
  - Komunikují pouze přes frontu (jinak o sobě vzájemně nevědí)

Implementace trvanlivosti fronty

- Nejjednodušší: tabulka v databázi
  - Časté přístupy tvoří **výkonnostní slabinu** (bottleneck)
- Vhodné: **oddělený aplikační modul** (message broker)
  - RabbitMQ, Apache ActiveMQ, Apache Kafka, Amazon SQS, …

Reálné události

- Některé akce transakce **nelze vrátit zpět**
  - Příklad: bankomat vydá hotovost → havárie → TPS vrátí DB, ale hotovost ne
- **Reálné (fyzické) události** (real-world events) nemají rollback
- Atomičnost musí být dosažena jinak → **dopředný návrat** + zotavitelná fronta

Řešení s frontami: reálná událost

- Transakce T **vloží požadavek** na fyzickou akci do fronty Q
- Pokud T je zrušena $\rightarrow$ požadavek odstraněn z Q
- Pokud T je potvrzena $\rightarrow$ požadavek zůstane v Q, fyzická akce se **jistě provede**
- Havárie po commitu: fronta přežije, akce bude provedena po restartu

Detekce stavu fyzické události

- Fyzické zařízení obsahuje **čítač** C inkrementovaný atomicky s provedením akce
- Transakce před commitem uloží aktuální hodnotu C do DB (D)
- Při zotavení z havárie: porovnání C a D:
  - $C = D \rightarrow$ fyzická akce nebyla provedena $\rightarrow$ provést znovu
  - $C \neq D \rightarrow$ fyzická akce provedena před havárií $\rightarrow$ odebrat záznam z fronty

Shrnutí: cesta k workflow

- Zotavitelné fronty umožňují **sekvence i paralelismus** transakcí
- Tvoří základ pro složitější **workflow systémy**
- Klíčové vlastnosti:
  - Trvanlivost plánovaných akcí
  - Oddělení producenta a konzumenta v čase
  - Podpora paralelního zpracování nezávislých větví

Jakarta Messaging a reaktivní mikroslužby

JMS, @MessageDriven, MicroProfile Reactive Messaging

Jakarta Messaging (JMS)

- Standardní Jakarta EE API pro práci s **message brokery** (balíček `jakarta.jms`)
- Jednotné rozhraní pro různé brokery: ActiveMQ Artemis, IBM MQ, Oracle AQ, …
  - Adaptér pro RabbitMQ
- Dva komunikační modely:
  - **Point-to-Point (Queue)** — zpráva jde jednomu konzumentovi
    - = zotavitelná fronta
  - **Publish/Subscribe (Topic)** — zpráva jde všem přihlášeným konzumentům
- Realizuje "oddělený aplikační modul (message broker)"

JMS XA transakce — řešení fyzické události

- Problém: atomické provedení **"ulož do DB"** + **"vlož do fronty"**
- Řešení: **JMS XA** — broker se chová jako XA resource, koordinovaný JTA (2PC)

Výsledek transakce	Co se stane se zprávou
Rollback	Zpráva se **neodešle** — záznam odstraněn z fronty
Commit	Zpráva **jistě doručena** — přežije restart systému

- Odpadá nutnost explicitního čítače pro detekci stavu fyzické události

JMS XA transakce — příklad producenta

```java
@Stateless
public class ObjednavkaService {

    @PersistenceContext
    EntityManager em;

    @Resource(lookup = "java:app/jms/ConnectionFactory")
    ConnectionFactory cf;

    @Resource(lookup = "java:app/jms/ExpediceQueue")
    Queue expediceQueue;
```

- `ConnectionFactory` musí být **XA-aware** (např. `ArtemisXAConnectionFactory`)

@MessageDriven — EJB konzument

```
@MessageDriven(activationConfig = {
    @ActivationConfigProperty(
        propertyName  = "destination",
        propertyValue = "java:app/jms/ExpediceQueue")
})
public class ExpediceMDB implements MessageListener {

    @Override
    @Transactional
    public void onMessage(Message msg) {
        long objednavkaId = msg.getBody(Long.class);
        // zpracuj expedici — v nové transakci
```

- `@MessageDriven` bean (MDB) = EJB konzument zotavitelné fronty
- Kontejner volá `onMessage()` pro každou zprávu asynchronně

MicroProfile Reactive Messaging — ukázka

```java
@ApplicationScoped
public class ObjednavkaProducer {

    @Inject
    @Channel("objednavky-out")
    Emitter<Long> emitter;

    public void vytvorObjednavku(Objednavka o) {
        em.persist(o);
        emitter.send(o.getId());   // async odeslání na kanál
    }
}
```

- Kanál je namapován na konkrétní broker (Kafka, RabbitMQ, …) v konfiguraci
- `@Outgoing` / `@Incoming` odpovídají rolím producenta/konzumenta fronty

JMS vs. MicroProfile Reactive Messaging

Aspekt	Jakarta Messaging (JMS)	MP Reactive Messaging
Threading	Blokující	Neblokující (Reactive Streams)
Typický kontext	Jakarta EE aplikace	Mikroslužby
Transakce	JTA (XA)	Závisí na brokeru a konfiguraci
Konzument	`@MessageDriven`	CDI bean s `@Incoming`

- Obě řeší **stejný problém**: asynchronní zpracování zpráv přes broker
- MP Reactive Messaging: viz přednáška p05

A to je vše!

Dotazy?